AUTOMATED AQI PREDICTOR

A Machine Learning-Based System for Proactive, Multi-Day Air Quality Index Forecasting

A Project Report Submitted in Fulfillment of the Requirements for the

10Pearls 10pShine Internship Program

Cohort 9

Submitted By:

Nazaar Khalid

TABLE OF CONTENTS

EXECUTIVE SUMMARY

Islamabad's seasonal air pollution is a well-known health hazard, yet most public weather tools only tell people how bad the air is *right now*. The AQI Predictor was developed to change that by offering a proactive, 72-hour forecast of PM2.5 concentrations, giving residents the foresight needed to plan their outdoor activities safely.

What sets this platform apart from standard weather dashboards is its focus on human-centric communication rather than raw data. Instead of displaying a number and leaving users to interpret it, the system uses generative AI to translate current air quality into a "cigarette-equivalent" warning, putting an otherwise invisible threat into terms people immediately understand. The platform also flips the usual browsing model: instead of requiring users to check the site every day, it evaluates upcoming forecasts on its own and sends an automated email the moment hazardous conditions are predicted for the following day.

Behind the scenes, the product runs on a zero-cost, serverless architecture. All of the heavy lifting — fetching weather data, running the machine learning models, and generating AI-written insights — happens entirely in the background. That separation keeps the client-facing dashboard fast and lightweight, no matter how much processing is happening behind it.

In short, the AQI Predictor turns a complex environmental modeling problem into something usable day-to-day: a machine learning system packaged simply enough that people can act on it before the smog arrives, not after.

CHAPTER 1 — PROJECT PROGRESS & DEVELOPMENT LIFECYCLE

The AQI Predictor moved through six phases, from early data exploration to a fully deployed web application. Working iteratively meant the core machine learning models could be validated thoroughly before any of the automation, infrastructure, or dashboard work began.

1.1 Phased Development Approach

The project followed an iterative methodology rather than a rigid, linear schedule, advancing through six milestones that culminated in a simultaneous full-stack deployment.

- **Phase 1: Experimentation & Prototyping:** Conducted within Google Colab to evaluate both the OpenWeatherMap and Open-Meteo APIs, perform Exploratory Data Analysis (EDA), engineer time-series features, and establish baseline model performance.
- **Phase 2: Infrastructure Foundation:** Transitioned from isolated scripts to a production database by provisioning Supabase (PostgreSQL) and designing a normalized schema for weather and forecast data.
- **Phase 3: Pipeline Automation:** Engineered the core CI/CD orchestration utilizing GitHub Actions to schedule unattended feature extraction and inference generation.
- **Phase 4: Backend Services:** Developed a scalable API layer using the Django REST Framework to securely query and serve database records and model inferences to client applications.
- **Phase 5: Frontend Implementation:** Built a responsive Single Page Application (SPA) using Vite and React, integrating Recharts to visualize real-time air quality forecasts and health alerts locally.
- **Phase 6: Production Deployment:** Deployed the Django API as a Web Service and the React SPA as a Static Site on Render, connecting the two through shared environment variables.

1.2 Phase 1 — Experimentation Phase

1.2.1 Environment & Tooling

The initial research and prototyping phase was executed within Google Colab to establish the baseline machine learning architecture before migrating to VS Code. The geographic scope of the model was strictly defined for Islamabad, Pakistan.

1.2.2 Data Source Evaluation

To build a 5-year historical training dataset, a hybrid API architecture was used: OpenWeatherMap supplied historical PM2.5 concentrations, while Open-Meteo provided the corresponding meteorological parameters. The models were built to predict raw PM2.5 concentrations directly rather than a qualitative AQI category — a custom function then converts those particulate predictions into standard US EPA AQI values as a separate post-processing step during inference.

1.2.3 Exploratory Data Analysis (EDA)

Analysis of the initial dataset revealed critical atmospheric behaviors dictating the modeling strategy:

- **Severe Multicollinearity:** Relative humidity and temperature displayed a strong negative correlation ($r = -0.95$), indicating that linear models would struggle.
- **Non-Linear Dispersion:** Wind speed exhibited a step-function threshold; velocities exceeding 17.5 m/s physically prevented localized pollutant accumulation, pulling the AQI below 100.
- **Bimodal Temperature Behavior:** Peak AQI spikes (130–190) occurred during mild dry-spell inversions, whereas peak summer temperatures were often accompanied by monsoon rain scrubbing that lowered the AQI.

1.2.4 Feature Engineering

To map atmospheric dynamics, a standardized feature matrix of 12 distinct variables was engineered for the machine learning models:

- **Pollution Memory:** A 24-hour lag (pm25_lag_24h) and a 24-hour rolling average (pm25_rolling_24h) were computed to provide models with baseline accumulation context.

- **Future Weather Alignment:** Meteorological variables (temperature, humidity, pressure) were explicitly shifted forward in time to align exactly with the target forecasting horizon (future_temp, future_humidity, etc.).
- **Wind Vector Decomposition:** Wind speed and direction were mathematically converted into separate horizontal and vertical components (wind_u, wind_v) using trigonometric functions.
- **Cyclical Time Encodings:** Sine and cosine transformations were applied to the hour (hour_sin, hour_cos) and month (month_sin, month_cos) to preserve chronological continuity.

1.2.5 Model Training

To establish a solid baseline, the initial training phase evaluated four different models on a short-term, 1-hour forecasting window. The goal was to find a model that maintained low error rates (MAE and RMSE) while maximizing the R^2 score. Finding this balance was important: MAE and RMSE show how close the predictions are to the actual PM2.5 values on average, while a high R^2 proves the model actually recognizes the patterns behind sudden pollution spikes instead of just playing it safe and guessing the historical average.

Table 1.1: Candidate Models Evaluated in Experimentation Phase

Model	MAE	RMSE	R^2
XGBoost	4.19	8.89	0.9473
Random Forest	4.36	9.70	0.9372
Ridge Regressor	6.17	11.6	0.9092
TensorFlow (Sequential)	7.36	11.86	0.9061

XGBoost outperformed the other baseline models by a clear margin, capturing the nonlinear weather patterns the alternatives struggled with. After selecting XGBoost as the core predictive engine, the architecture was expanded into three separate models using the full feature set to predict across extended multi-day horizons.

- **Day 1 (+24h):** Achieved an R^2 of 0.8244.

- **Day 2 (+48h):** Achieved an R^2 of 0.6295.
- **Day 3 (+72h):** Achieved an R^2 of 0.4955.

1.2.6 Moving to VS Code

With the data validated and the models trained in Google Colab, the experimentation phase came to an end. The workflow was moved locally to VS Code to start developing the full application, including the database setup, automated pipeline scripts, API backend, and frontend dashboard.

1.3 Phase 2 — Infrastructure Foundation

1.3.1 Database Platform Selection

Hopsworks was initially evaluated as the primary repository for both engineered features and trained models, but it became clear during development that the free tier wasn't built to handle continuous, hourly relational data ingestion at scale. The architecture was split as a result: Hopsworks was kept strictly as a Model Registry for the trained .pkl artifacts, while Supabase — a managed PostgreSQL platform — took over as the primary relational database. Supabase's native handling of time-series data and its straightforward integration with Python's SQLAlchemy meant the automated pipelines could push data into tables directly using Pandas DataFrames.

1.3.2 Schema Design

Table 1.2: Core Database Schema Overview

Table Name	Purpose	Key Columns
aqi_features	Stores the historical weather and PM2.5 feature matrix for model training and inference.	datetime, pm2_5_ugm3, temp_celsius, wind_u, wind_v, ai_insight
predictions_aqiforecast	Persists the multi-horizon predictions generated by the inference pipeline.	created_at, target_time, target_horizon, predicted_pm25
model_metrics	Stores the performance evaluation metrics and	horizon, model_type, r2, mae, rmse, shap_data

Table Name	Purpose	Key Columns
	SHAP feature importance data for dashboard visualization.	
alerts_subscriber	Manages user registrations for automated high-AQI email notifications.	email, is_active

1.4 Phase 3 — Data & ML Pipelines

1.4.1 Automated Feature Extraction Pipeline

To maintain a continuous flow of live data, the feature extraction process was automated into a scheduled pipeline using GitHub Actions. The script queries the OpenWeatherMap API for recent historical air pollution metrics and the Open-Meteo API for corresponding meteorological data. To prevent redundant data ingestion, the pipeline queries the Supabase aqi_features table to identify the latest recorded timestamp and only processes rows that chronologically follow it.

During extraction, all of the required machine learning features (e.g., wind vector decomposition, 24-hour rolling averages, and cyclical time transformations) are computed. The pipeline also calls the Gemini API (gemini-3.5-flash) to generate a conversational health insight: the script first calculates the PM2.5 cigarette-equivalence (PM2.5 / 22.0) in Python, then prompts the LLM to wrap that number in a punchy, two-sentence alert. Both the computed features and the AI-generated text are appended to the Supabase database.

1.4.2 Automated Model Training Pipeline

A dedicated training pipeline was built so the system could continuously learn from newly ingested data rather than relying on a static, manually trained model. It pulls the full historical feature dataset from Supabase, applies the time-shifts needed for each horizon, and retrains the Day 1, Day 2, and Day 3 XGBoost models using an 80/20 validation split.

Beyond training the models, the script integrates the shap library to calculate the relative importance of each weather feature in the decision-making process. Once training completes, the pipeline pushes the updated .pkl models to the Hopsworks Model Registry and writes the fresh R^2, MAE, RMSE scores, and SHAP data directly into the Supabase model_metrics table so the frontend dashboard reflects the current model health.

1.4.3 Automated Inference Generation Pipeline

Following the feature extraction, the inference pipeline executes to generate future predictions. The script pulls the most recent 25 hours of foundational data from Supabase alongside a fresh 72-hour future weather forecast from Open-Meteo.

The pipeline authenticates with the Hopsworks Model Registry to dynamically download the latest version of the trained XGBoost models (day_1, day_2, and day_3). It constructs the necessary future feature matrices, generates PM2.5 predictions for each horizon, and saves the results into the predictions_aqiforecast table.

This pipeline also doubles as the system's monitoring agent. It converts the Day 1 (+24h) PM2.5 prediction into a standard AQI value; if the forecasted AQI exceeds the threshold of 150 (Unhealthy), the script queries the alerts_subscriber table and automatically dispatches warning emails via SMTP to all active subscribers, advising them of the impending air quality degradation.

1.5 Phase 4 — Backend Services

1.5.1 Django API Design & Routing

The backend API was developed using Django and the Django REST Framework to serve as a bridge between the Supabase database and the client application. To keep the database clean, the API dynamically converts raw PM2.5 readings into standard US EPA AQI values at the exact moment the data is requested, rather than storing redundant AQI numbers in the database tables.

To minimize frontend load times and reduce the number of network requests, the primary dashboard endpoint was designed to aggregate current conditions, 24-hour historical data, and 72-hour future forecasts into a single, consolidated JSON response. Additionally, the backend explicitly converts all UTC database timestamps to the local Asia/Karachi timezone to ensure the forecast times render accurately for end-users.

Table 1.3: Core API Endpoints

Endpoint	Method	Purpose	Response Format
/api/dashboard/	GET	Aggregates recent weather history, current PM2.5/AQI, AI insights, and the next 72 hours of localized forecasts.	JSON
/api/model-metrics/	GET	Executes a raw SQL cursor query to fetch dynamically generated model R^2, MAE, RMSE, and SHAP feature importance data.	JSON
/api/subscribe/	POST	Accepts user email submissions and validates uniqueness to register users for high-AQI automated alerts.	JSON

1.6 Phase 5 — Frontend Implementation

1.6.1 Application Scaffolding

The client-side interface was built as a Single Page Application (SPA) utilizing React and Vite. To optimize rendering speed, the application state is managed centrally within the App.jsx component. On initial load, a single Axios request is made to the Django backend to fetch the consolidated dashboard payload. This data is then passed down as props to the modular components (Dashboard, Developer, and About), allowing users to navigate between the main forecast view and the backend model diagnostics without triggering page reloads.

1.6.2 HCI & Interface Design Choice

A significant focus of the frontend development was ensuring strict Human-Computer Interaction (HCI) compliance to maximize usability and comfort. The application layout is structured strictly by user priority: immediate health threats and current AQI are positioned at the top, followed by AI-generated contextual insights, actionable behavioral recommendations (e.g., outdoor exercise safety), and finally, the extended multi-day forecasts.

To provide immediate, intuitive visual feedback without requiring the user to read the exact numbers, a custom ParticleBackground canvas was engineered. This component renders ambient floating particles that dynamically increase in density and shift in color (from safe greens to hazardous purples/reds) based on the live AQI, instantly conveying the threat level. Finally, subtle localized aesthetics were embedded into the design; for example, the curved

black shape of the application logo was designed as a minimalist nod to the Margalla Hills, a defining geographical feature of Islamabad.

1.6.3 Data Visualization

To make the complex time-series data digestible, the Recharts library was integrated to build responsive, interactive data visualizations. On the main dashboard, a 24-hour historical line chart tracks recent pollution momentum. This chart utilizes an SVG linear gradient that automatically shifts the line color from green to red the moment the data crosses the threshold into unsafe AQI territory.

Additionally, a dedicated "Developer" view was built to visualize the machine learning diagnostics. This view renders the R^2, MAE, and RMSE metrics dynamically and includes a horizontal bar chart mapping the SHAP feature importance, allowing developers to visually audit exactly which meteorological variables are currently driving the XGBoost predictions for each distinct forecasting horizon.

1.7 Phase 6 — Deployment

1.7.1 Backend Deployment

The Django API was deployed on Render as a Web Service. Environment variables were centralized within Render's dashboard to securely connect the live application to the production Supabase database. A major architectural advantage of this deployment strategy is its operational efficiency. Because the heavy computational lifting—such as historical data extraction, feature engineering, model training, and XGBoost machine learning inference—is completely offloaded to scheduled GitHub Actions pipelines, the deployed backend stays lightweight by comparison. The Django server is left to handle only simple read queries against Supabase and serve JSON payloads, which keeps response latency low even as the underlying models grow more complex.

1.7.2 Frontend Deployment

The React Single Page Application (SPA) was deployed concurrently on Render as a Static Site. The build process was configured to inject the live backend URL via the VITE_API_BASE_URL environment variable, enabling seamless communication between the two distinct servers. Serving the frontend as a static site keeps initial load times fast for the client dashboard. Combined with the low-latency JSON responses from the decoupled

backend, this keeps the application responsive, delivering visual feedback and health alerts to the user with minimal delay.

CHAPTER 2 — SYSTEM ARCHITECTURE

2.1 High-Level System Flow

The AQI Predictor operates as a fully decoupled, event-driven architecture. The data lifecycle begins with scheduled GitHub Actions workflows acting as the primary orchestration engine. These pipelines extract live meteorological and pollution data from Open-Meteo and OpenWeatherMap, compute the required machine learning features, and persist the clean datasets into a Supabase PostgreSQL database. Concurrently, the inference pipeline dynamically fetches the latest XGBoost models from the Hopsworks Model Registry, generates 72-hour PM2.5 forecasts, logs them to Supabase, and dispatches SMTP email alerts if hazardous conditions are detected.

On the client side, the system employs a pull-based architecture. When a user accesses the dashboard, the React Single Page Application requests data from the Django REST API. The Django backend queries the Supabase instance, aggregates the historical data, AI-generated insights, and future forecasts, and returns a consolidated JSON payload. This separation ensures that the intensive machine learning workloads never block or slow down the web-serving infrastructure.

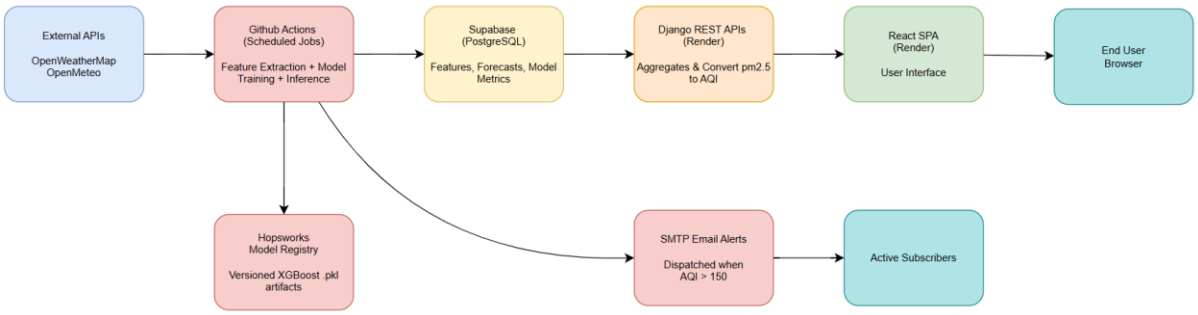

Figure 2.1: High-Level System Architecture Diagram

2.2 Technology Stack Breakdown

While the original project guidelines suggested using Flask and Streamlit, Django and React were selected instead. Both frameworks offered greater existing familiarity and significant advantages for a production build. Streamlit is well suited to quick prototypes, but React provided complete control to build a highly responsive, custom interface such as the interactive

particle background. Similarly, Django's built-in tools made managing secure database connections and API routing considerably smoother than Flask would have allowed.

A core requirement in the project guidelines was maintaining a serverless architecture. The data storage strategy evolved to meet this while solving data capacity constraints. Hopsworks was initially planned as the sole data store, but it struggled to handle the volume of continuous hourly weather and pollution records. Pivoting to Supabase allowed the system to remain fully serverless, as Supabase provides a managed, cloud-native PostgreSQL environment requiring zero manual server maintenance, while Hopsworks was retained purely to store and version the trained machine learning models.

Table 2.1: Full Technology Stack by Layer

Layer	Technology	Role
Data Source	OpenWeatherMap / OpenMeteo	Source of ground-truth PM2.5 and meteorological data.
Automation / Orchestration	GitHub Actions	Executes automated cron jobs for feature extraction and ML inference.
Database	Supabase (PostgreSQL)	Primary persistent relational storage for features, forecasts, and metrics.
Model Registry	Hopsworks	Centralized cloud storage for versioned XGBoost .pkl artifacts.
Machine Learning	Python, XGBoost, SHAP	Trains predictive models and calculates feature attribution.
AI Integration	Google Gemini API (3.5-Flash)	Generates dynamic, conversational health alerts based on current PM2.5.
API Layer	Django + DRF	Serves as the secure data-fetching layer bridging the DB and frontend.
Frontend	React + Vite + TailwindCSS	Renders the responsive, HCI-compliant client dashboard.
Data Visualization	Recharts	Powers the interactive AQI trend lines and SHAP diagnostic bar charts.
Version Control	Git / GitHub	Source control, CI/CD trigger

Layer	Technology	Role
Hosting	Render	Hosts both the Django Web Service and the static React site

2.3 Deployment Topology

The system's infrastructure is spread across a few different cloud platforms to balance performance and keep costs low. The heavy lifting—like running the data pipelines and generating model predictions—is handled automatically by GitHub Actions. All the actual data and models are stored securely in Supabase and Hopsworks. Finally, the Django backend and React frontend are hosted live on Render. Because these pieces are separated, they communicate entirely through secure API calls and database connections. This setup keeps the application fast, reliable, and easy to scale up in the future.

CHAPTER 3 — CORE SYSTEM FEATURES

The AQI Predictor packages complex atmospheric data and machine learning predictions into features people can act on day to day.

3.1 End-User Dashboard & Visualization

The primary interface is engineered for rapid information retrieval, prioritizing immediate health threats while offering deep data exploration.

- **Real-Time Environmental Metrics:** A live display of the standardized US EPA AQI, alongside current temperature and humidity.
- **Dynamic Visual Feedback:** The UI features a custom particle background that dynamically adjusts in density and color—shifting from safe greens to hazardous purples and reds—letting users gauge the threat level at a glance without parsing exact numbers.
- **Actionable Health Recommendations:** Conditional visual indicators immediately advise users on the safety of outdoor exercise, general outdoor exposure, and whether it is safe to keep windows open.
- **Interactive 24-Hour Trend:** A responsive historical line chart tracks recent pollution momentum. It utilizes a dynamic SVG gradient that automatically maps the line color to corresponding safe or unsafe AQI threshold zones.

3.2 Predictive Forecasting & AI Insights

The system moves beyond historical tracking to give users a look ahead, not just a snapshot of current conditions.

- **72-Hour Multi-Horizon Outlook:** Powered by an XGBoost machine learning model, the dashboard provides distinct, localized AQI forecasts for Day 1 (+24h), Day 2 (+48h), and Day 3 (+72h).
- **Generative AI Health Alerts:** Integrated with Google's Gemini 3.5-Flash model, the system converts raw PM2.5 measurements into conversational, easily digestible health warnings. It mathematically calculates the cigarette-equivalent of breathing the current air and generates a punchy, two-sentence alert to contextualize the severity of the pollution.

3.3 Automated Alerting System

To ensure users remain protected even when they are not actively checking the dashboard, a proactive alerting mechanism was implemented.

- **Frictionless Subscription:** A streamlined form allows users to easily register their email addresses for environmental warnings.
- **Threshold-Triggered Notifications:** Operating autonomously via the backend pipeline, the system evaluates the Day 1 (+24h) forecast. If the predicted AQI exceeds 150 (Unhealthy), SMTP email warnings are automatically dispatched to all active subscribers, providing advanced notice to adjust outdoor plans.

3.4 Developer & Diagnostic Tools

A dedicated diagnostic view was built specifically for the evaluation team and developers to ensure the underlying machine learning architecture remains transparent and auditable in production.

- **Live Model Analytics:** Real-time visibility into the forecasting engine's performance, displaying current R^2, MAE, and RMSE scores for each forecasting horizon.
- **SHAP Feature Attribution:** A horizontal bar chart maps the Shapley Additive exPlanations (SHAP) values, allowing developers to visually audit exactly which meteorological variables (e.g., wind vectors, pressure, or historical lag) are driving the AI's current predictions.

CHAPTER 4 — MODEL DIAGNOSTICS & PERFORMANCE

4.1 Validation

To accurately evaluate the XGBoost models on the 5-year dataset, an 80/20 chronological split was strictly enforced during the training pipeline. Traditional random train-test splitting was avoided because it would cause "data leakage" (allowing the model to peek at future weather events to predict past pollution). To prevent the models from simply memorizing the training data, the tree architecture was constrained using deliberate hyperparameters: a shallow tree depth (max_depth=6), a slow learning rate (learning_rate=0.04), and a fixed number of boosting rounds (n_estimators=200).

4.2 Multi-Horizon Performance Metrics

The final models deployed to the dashboard exhibit a clear, expected decay in accuracy as the forecasting horizon extends further into the future.

- **Day 1 (+24h):** The model performs strongly, achieving an R^2 of 0.84, with a Mean Absolute Error (MAE) of 20.95 µg/m³ and a Root Mean Squared Error (RMSE) of 30.26 µg/m³.
- **Day 2 (+48h):** As weather uncertainty increases, the R^2 drops to 0.6, with an MAE of 34.1 µg/m³ and an RMSE of 48.35 µg/m³.
- **Day 3 (+72h):** The longest horizon yields an R^2 of 0.45, an MAE of 40.49 µg/m³, and an RMSE of 56.69 µg/m³.

4.3 Feature Importance (SHAP Analysis)

The SHAP (Shapley Additive exPlanations) charts generated in the developer dashboard reveal exactly how the model shifts its logic depending on the time horizon.

For the Day 1 forecast, short-term pollution momentum is by far the dominant factor in the prediction. The 24-hour rolling average (pm25_rolling_24h) and the immediate PM2.5 reading (pm2_5_ugm3) are the strongest drivers by a wide margin. However, by Day 2, the model shifts: immediate pollution levels are no longer as reliable, and the cyclical month encoding (month_cos) overtakes the momentum features as the most impactful variable. By Day 3, short-term memory is almost entirely stripped away, and the model relies heavily on macro-seasonality (month_cos) to estimate baseline pollution accumulation.

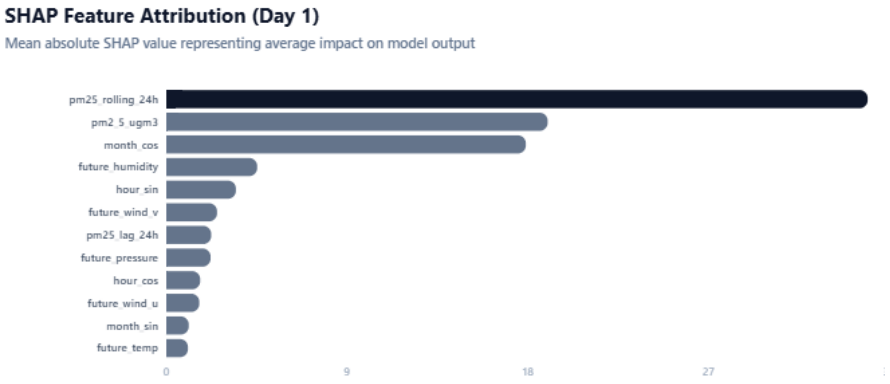

Figure 4.1: SHAP Feature Importance Day 1

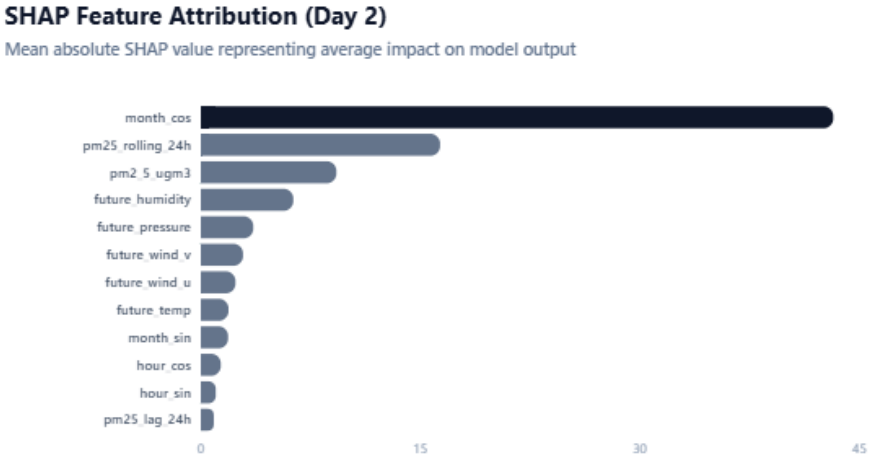

Figure 4.2: SHAP Feature Importance Day 2

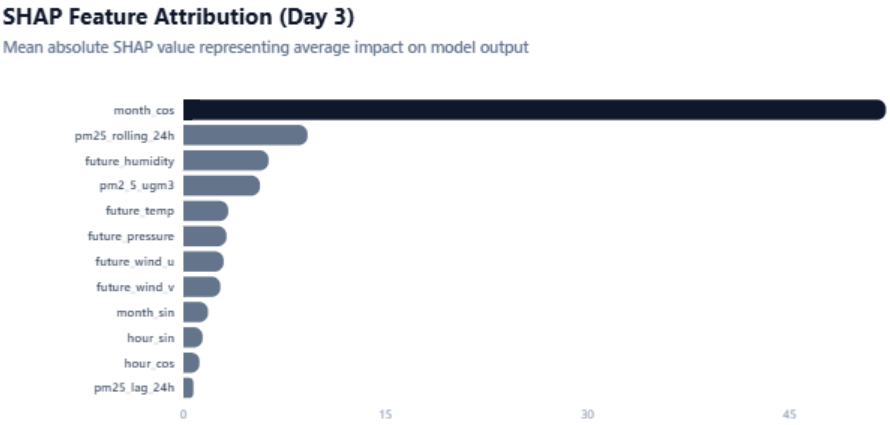

Figure 4.3: SHAP Feature Importance Day 3

CHAPTER 5 — CHALLENGES & STRATEGIC MITIGATIONS

5.1 Challenge Summary Table

The development lifecycle presented several architectural and data-modeling hurdles. Table 5.1 outlines the primary challenges encountered, from initial experimentation to final deployment, alongside their respective mitigation strategies.

Table 5.1: Challenges Encountered and Resolutions

#	Challenge	Root Cause	Mitigation Strategy	Outcome
1	Wind Direction Cyclicality	Raw wind degrees (0° and 360° are identical) broke XGBoost tree-splitting logic, causing inaccurate weather physics representation.	Mathematically decomposed raw wind direction into independent horizontal and vertical vectors (wind_u, wind_v).	The model accurately captures continuous wind trajectories without artificial discontinuities.
2	Chronological Data Leakage	Random train/test splits allowed the model to look at future weather events to predict past pollution, creating artificially high accuracy.	Enforced a strict 80/20 sequential slice based entirely on time, completely abandoning random splitting.	Ensured realistic evaluation metrics that accurately reflect true real-world forecasting limitations.
3	Database Ingestion Bottlenecks	Hopsworks was optimized for model artifacts, struggling to sustain the high-frequency ingestion of hourly time-series weather data, a constraint likely attributable to the limits of its free-tier service.	Pivoted primary relational storage to Supabase (PostgreSQL), retaining Hopsworks solely for versioning the .pkl models.	Achieved a stable, low-latency automated data ingestion architecture.
4	Pipeline Race Conditions	Independent GitHub Actions cron schedules	Replaced independent timers	Eradicated race conditions, ensuring

#	Challenge	Root Cause	Mitigation Strategy	Outcome
		allowed the inference pipeline to execute before feature extraction finished, generating forecasts on stale data.	with explicit workflow_run triggers, chaining the pipelines for strict sequential execution.	predictions are always generated using the absolute latest features.
5	Timezone Desynchronization	Database timestamps defaulted to UTC, resulting in the client dashboard displaying incorrect prediction hours for local users.	Implemented pytz in the Django views to dynamically cast all timestamps to Asia/Karachi directly during API serialization.	Localized UI displaying highly accurate, perfectly time-synced predictions for Pakistani users.

5.2 Other Notable Challenges

- **Redundant Data Ingestion:** Scheduled extraction pipelines risked inserting duplicate weather records if APIs were queried blindly. This was mitigated by having the script query Supabase for the most recent stored timestamp before pulling new Open-Meteo data, appending only the missing delta.
- **LLM Pipeline Stability:** Relying on the Gemini API to autonomously calculate and generate health insights risked numerical hallucinations. This was mitigated by calculating the exact cigarette-equivalence mathematically in Python first, then strictly prompting the LLM to wrap that hard value in conversational text.

CHAPTER 6 — CONCLUSION & FUTURE ENHANCEMENTS

6.1 Conclusion

The AQI Predictor turns raw environmental data into a tool people can actually act on. The original goal was to move past simply reporting historical pollution levels, and the finished 72-hour forecasting dashboard does exactly that.

Looking back, the decision to build on a fully decoupled, serverless architecture paid off. Pushing the heavy machine learning workloads into automated background pipelines kept the live application fast and reliable without needing expensive infrastructure. The finished system is a solid proof-of-concept that predictive models can be delivered in a lightweight, user-first package — and by prioritizing visual, immediate feedback over dense numbers, it gives people what they actually need to plan their day around the air they'll be breathing.

6.2 Future Enhancements

- **Multi-City Expansion:** Expand the backend pipelines to support other major urban centers across Pakistan from a single unified dashboard.
- **Personalized Alerts:** Add basic user accounts so sensitive individuals can set custom AQI email notification thresholds instead of relying on the default system-wide trigger.
- **Confidence Intervals:** Provide upper and lower probabilistic bounds alongside the point predictions, making the inherent uncertainty of the Day 2 and Day 3 forecasts transparent to the user.

6.3 Personal/Professional Reflection

Building this project was a huge learning curve. It made me realize that shipping a real software product takes a lot more than just writing good code. A massive part of this project's success came down to early planning, specifically, taking the time to run an initial experimental phase to test the XGBoost models and finalize my features before ever touching the final frontend.

It also required a lot of on-the-fly strategizing. Trying to build a robust machine learning pipeline on a strictly zero-cost, serverless architecture really taught me how to play the hand I was dealt. When initial database plans didn't work out, I had to pivot quickly and work

creatively around hard technical constraints. Above all, this project shifted how I think about user experience. Instead of just dumping raw PM2.5 numbers on a screen, I focused heavily on visual feedback, like the dynamic background colors, so people could instantly recognize the threat level. It was a challenging process, but it completely changed how I approach building software products from end to end.